

Shover World - Millstone 1 - Report

Environment and GUI

Mahla Entezari*

Shahid Beheshti University, Tehran, Iran
Artificial Intelligence course – Fall 2025

November 22, 2025

I. INTRODUCTION

Shover-World is a small grid-based environment designed for experimenting with decision-making and planning in artificial intelligence. Even though the world is simple, it contains several interesting methods that make the environment more than a basic movement task and help us understand how agents must think before taking actions.

A Pygame-based graphical interface was also developed to visualize the world. The GUI allows users to see changes on the grid in real time, making it much easier to understand how the environment evolves step by step.

II. BACKGROUND

Many AI problems use grid environments because they show exactly how an agent interacts with its surroundings. A structure should provide clear rules for observations, actions, and how the world changes after each step. This structure allows us to test ideas in a controlled and visible way.

In Shover-World, the agent must manage limited stamina, account for the effort needed to push objects, and sometimes perform special actions that can reshape the world. These features make the environment useful for studying more than simple navigation—they help highlight planning, resource usage, and recognition of spatial patterns.

III. PROBLEM DEFINITION AND RULES

The goal of Shover-World is to design a world where an agent interacts with objects rather than just moving around. Each step the agent takes has consequences, and the environment enforces rules that determine whether an action succeeds or fails.

A key mechanic is the **chain push**, where the agent attempts to push multiple boxes in a straight line. A push only succeeds if the space after the last box is free or contains lava. This creates a small puzzle each time the agent chooses to push.

The agent also has a limited amount of stamina. Ordinary movement always uses one stamina point. Pushing is more expensive and depends on how many boxes

are in the chain. Pushing the same chain in the same direction becomes cheaper, while pushing into lava destroys boxes and gives stamina back.

Another rule involves recognizing **perfect squares**—blocks of boxes arranged in an $n \times n$ shape with no extra boxes touching the border. Perfect squares can be transformed using special actions. The Barrier Maker turns them into solid barriers, and Helify converts their interior into lava.

All these rules together form a small but meaningful problem where the agent must think ahead, manage its stamina, and sometimes reshape the world to reach its goals.

IV. METHODOLOGY

A. Environment Architecture

The Shover-World environment is designed using the same structure as a standard Gym environment. This makes it easy to use, easy to test, and easy to connect to reinforcement learning algorithms in the future. A Gym-style environment usually includes:

- a `reset()` function to start a new episode,
- a `step()` function that applies one action,
- a clearly defined *observation space*,
- a clearly defined *action space*,
- and a `close()` function for cleanup.

*MahlaEntezari.sbu@gmail.com

Shover-World follows all these rules so that an RL agent can interact with it without any changes.

1. Core Class Structure

The environment is implemented as a Python class named `ShoverWorldEnv`. When creating an instance of this class, the user can provide several settings:

- grid size: number of rows and columns,
- number of boxes, barriers, and lava cells,
- initial stamina,
- special force values for pushing,
- an optional map file to load a custom grid.

If a map file is not provided, the environment creates a random map automatically. This makes the environment flexible for different experiments.

2. Internal State

The grid is stored as a NumPy array called `grid` with shape:

$$n_{\text{rows}} \times n_{\text{cols}}$$

Each cell contains an integer that represents empty space, a box, a barrier, or lava.

The agent's position is *not* stored in the grid. Instead, it is stored separately as:

$$\text{agent_pos} = (r, c).$$

This keeps the grid clean and makes movement logic easier.

The environment also tracks:

- **stamina** (energy the agent has left),
- **timestep** (how many moves have passed),
- **_last_pushed_head** (to calculate push costs),
- **_square_registry** (list of perfect squares and their ages).

3. Action Space

The action space is defined using:

```
spaces.Tuple((position, action_type))
```

Even though the position is not used for movement, it is included because the assignment requires it. The **action type** determines what the agent tries to do:

- 1–4: move $\uparrow, \rightarrow, \downarrow, \leftarrow$
- 5: Barrier Maker
- 6: Hellify
- 0: do nothing (NOOP)

4. Observation Space

The environment returns an observation dictionary containing the grid (as a `int32` array), the agent's position, current stamina, the last selected position, and the last action taken.

Everything is returned in Gym-compatible formats so that RL libraries can use the environment easily.

5. Step Function

The `step()` function is the heart of the environment. It performs many tasks in the correct order:

1. Read the action.
2. Move the agent or attempt a push.
3. Apply stamina costs.
4. Destroy boxes in lava if needed.
5. Detect all perfect squares.
6. Increase their ages.
7. Dissolve old squares automatically.
8. Apply special actions (5 or 6) if selected.
9. Build the next observation.

It returns:

(observation, reward, done, info)

The **info** dictionary includes extra values to check if the action was valid, how many boxes were pushed or destroyed, and the list of current perfect squares.

B. Grid Representation and Map Loading

To keep the environment efficient, the grid is stored as a NumPy array. Each object type is represented by a numerical value to make updates as fast as possible. The agent's position is kept separately so that moving it does not require editing the grid array.

Maps can be created in two ways. The environment can load a map from a file, or it can generate one randomly. Two file formats are supported. Format A contains numerical values for each cell, matching the internal representation exactly. Format B uses simple symbols, making it easier to edit by hand. The environment automatically checks for valid input, correct row lengths, and the presence of a valid starting position.

If no map file is used, the environment places the agent randomly and fills the grid with the required number of boxes, barriers, and lava cells. Random maps are helpful for testing and make gameplay more varied during manual use.

C. Agent Actions and Stamina Logic

The agent in Shover-World can perform several actions, and each action affects the environment in a different

way. This section explains how these actions work and how stamina is spent or recovered during interaction.

1. Available Actions

The environment supports the following action types:

- **Movement actions (1–4)** Move one cell in the direction:

Up, Right, Down, Left.

- **Action 5: Barrier Maker** Converts a perfect square into barrier cells.
- **Action 6: Hellify** Converts the inside of a perfect square into lava.
- **Action 0: No-op** The agent does nothing this step.

Each action is applied through the `step()` function, which updates the grid and the agent's stamina.

2. Movement and Chain Push

When the agent moves into an empty cell, the move succeeds with:

$$\text{Cost} = 1 \quad (\text{stamina point}).$$

If the agent moves into a box, it attempts a **chain push**. This involves collecting all consecutive boxes in the direction of movement (forming a chain of length k), checking the cell immediately after the chain, and if that cell is:

- empty $\rightarrow$ push succeeds,
- lava $\rightarrow$ push succeeds and destroys the last box,
- barrier or box $\rightarrow$ push fails.

A valid chain push shifts all boxes forward by one cell.

3. Stamina Cost Formula

The stamina system forces the agent to think carefully before pushing. The cost of pushing a chain of k boxes is:

$$\text{PushCost} = \begin{cases} F_0 + kF_u, & \text{new push direction} \\ kF_u, & \text{continues in same direction} \end{cases}$$

Where:

$$F_0 = \text{initial force}, \quad F_u = \text{unit force per box}.$$

This models the idea that “starting a push is harder than continuing it.”

4. Stamina Refund (Lava Interaction)

If the final box of a chain is pushed into a lava cell:

$$\text{Refund} = F_0,$$

and that amount of stamina is added back to the agent. This creates a strategic choice: Keep boxes for later use, or destroy them to gain energy.

5. Invalid Actions

An action is considered invalid if:

- the agent tries to move outside the grid,
- it tries to push into a barrier,
- or the push chain cannot move because of blocking cells.

Invalid actions:

do not move the agent, but still cost 1 stamina.

The combination of ordinary movement, chain pushing, special actions, and stamina costs creates a world where the agent must plan ahead instead of acting randomly.

D. Perfect Squares System

These squares are not just decorations—they affect the rules of the game, the agent's choices, and even the special actions.

1. What Counts as a Perfect Square?

A perfect square is a block of boxes arranged in a clean shape:

$$\text{Perfect Square} = n \times n \quad \text{where } n \geq 2$$

For example:

$$\begin{array}{cc} 10 & 10 \\ 10 & 10 \end{array} \quad \begin{array}{ccc} 10 & 10 & 10 \\ 10 & 10 & 10 \\ 10 & 10 & 10 \end{array}$$

Both of the above are valid perfect squares.

But there is one more important rule:

- The square must have **no boxes touching its border**.

Meaning the one-cell border around it must not include any box values. This avoids “accidental” squares that appear inside a messy pile of boxes.

2. How the Environment Detects Squares

The function `_find_perfect_squares()` checks the grid using two main steps:

1. **Check the inside:** The block from (r, c) to $(r + n - 1, c + n - 1)$ must be completely filled with box values:

$$\forall(i, j) \in \text{block}, \quad \text{grid}[i, j] \in [1, 10].$$

2. **Check the border:** The border from $(r - 1, c - 1)$ to $(r + n, c + n)$ must contain:

no box values.

If both checks pass $\rightarrow$ it is a perfect square.

This scanning happens every step, so the environment always knows which squares currently exist.

3. Square Registry

To keep track of how long each square has stayed on the grid, the environment uses a small “registry”. Each entry in the registry stores:

- n : the size of the square,
- (r, c) : its top-left corner position,
- age: how many steps it has survived.

Every time the agent takes a step:

1. All existing squares are found again.
2. Old ones get their age increased:

$$\text{age} \leftarrow \text{age} + 1.$$

3. Newly created squares get age 0.
4. Squares that disappeared are removed.

This system is similar to keeping a “birthday list” for every perfect square.

4. Dissolving Old Squares

When a square becomes too old, it automatically disappears. The environment checks:

$$\text{age} \geq \text{perf_sq_initial_age}.$$

If this condition is true:

$$\text{grid cells} \rightarrow \text{empty (0)}.$$

This behavior prevents the grid from filling with permanent blocks and encourages the agent to act before squares vanish.

Perfect squares are important because:

- * They trigger special actions(Barrier Maker and Hellify)
- * They create structure in the grid,

- * They add time-based strategy,
- * They reward organized box pushing,
- * They make the world more interesting than random piles of boxes.

Without perfect squares, the environment would only be about pushing boxes. With them, the agent must think about patterns, timing, and map transformation.

The perfect square system brings organization, timing, and strategy into Shover-World. It allows the environment to detect patterns, age them, and dissolve them, giving the agent meaningful opportunities for smart planning and use of special abilities.

E. Special Actions: Barrier Maker and Hellify

These actions allow the agent to change whole regions of the grid at once. Both actions only work when a *perfect square* of boxes exists. If no perfect square is available, the action simply does nothing.

1. Barrier Maker (Action 5)

The Barrier Maker turns an entire perfect square of boxes into solid walls. The steps for this action can be described as:

1. Find all perfect squares in the grid.
2. If none exist, mark the action as invalid.
3. Otherwise, choose a square using the rule:
pick the smallest, and if tied, the oldest one.
4. Convert the entire $n \times n$ area to barrier cells:

$$\text{grid}[r : r + n, c : c + n] \leftarrow 100.$$

5. Give the agent a stamina reward:

$$\Delta \text{stamina} = n^2.$$

A small example:

$$\begin{bmatrix} 10 & 10 \\ 10 & 10 \end{bmatrix} \xrightarrow{\text{Barrier Maker}} \begin{bmatrix} 100 & 100 \\ 100 & 100 \end{bmatrix}$$

This action is useful when the agent wants to:

- block a corridor,
- stop boxes from moving into an area,
- or simply gain stamina.

Because barriers cannot be moved or destroyed, the agent must think carefully before using this action.

2. *Hellify (Action 6)*

The Hellify action works very differently. Instead of creating barriers, it creates a dangerous lava pit. This action requires:

$$n > 2,$$

because a 2×2 square has no “inside” area.

For a chosen perfect square:

- The **border** becomes empty:

$$\text{border cells} \leftarrow 0$$

- The **interior** becomes lava:

$$\text{inner cells} \leftarrow -100$$

Example for a 3×3 square:

$$\begin{bmatrix} 10 & 10 & 10 \\ 10 & 10 & 10 \\ 10 & 10 & 10 \end{bmatrix} \xrightarrow{\text{Hellify}} \begin{bmatrix} 0 & 0 & 0 \\ 0 & -100 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Boxes inside the lava area are instantly destroyed. This can be useful when the agent wants to clear a large group of boxes quickly.

The two special actions make the game world more dynamic by allowing the agent to alter the map itself. This adds several interesting choices:

- * Should the agent destroy boxes using lava, or save them for pushing?
- * Should the agent create barriers to block enemies (in future versions)?
- * Should the agent use squares immediately or wait for them to grow older?
- * Could the agent accidentally trap itself by building barriers in the wrong place?

Together, these actions turn Shover-World into a world where planning and timing matter just as much as movement.

F. *GUI Architecture*

While the environment can run without any graphics, the GUI makes testing, debugging, and playing much easier. In this section, I explain how the GUI is organized, how it talks to the environment, and how a human player can control the agent.

1. *Using Pygame for Graphics*

The GUI is built using the Pygame library. Pygame is a simple tool for making games. It provides window creation, drawing rectangles and text, reading keyboard and mouse input, and also refreshing the screen

at a steady frame rate.

The window size depends on the grid size. If the grid is $n_{\text{rows}} \times n_{\text{cols}}$, then the GUI computes a suitable cell size:

$$\text{cell_size} = \frac{\text{window_size}}{\max(n_{\text{rows}}, n_{\text{cols}})}$$

so that the entire map always fits on the screen.

2. *Heads-Up Display*

Above the grid, the GUI shows important information, including:

- timestep number,
- stamina left,
- number of boxes remaining,
- number of boxes destroyed,
- whether the last action was valid,
- how many boxes were destroyed in the most recent push.

All of this information comes directly from the environment’s **info** dictionary.

3. *Keyboard Controls*

The player can control the agent in real time. The GUI listens for key presses:

Key	Action
W / Up Arrow	Move Up (Action 1)
D / Right Arrow	Move Right (Action 2)
S / Down Arrow	Move Down (Action 3)
A / Left Arrow	Move Left (Action 4)
B	Barrier Maker (Action 5)
H	Hellify (Action 6)
R	Reset environment
Q	Quit program

Every time the player presses a key, the GUI sends an action and then redraws the new state.

4. *Mouse Controls*

The GUI also supports mouse input. When the player clicks on a cell:

$$\text{mouse_pos} \rightarrow (r, c)$$

the agent moves directly to that position *if it is not blocked by a barrier*. This is helpful for testing because it lets the user jump across the map quickly.

5. *Main Event Loop*

The GUI runs inside a loop that repeats until the player exits:

1. Read keyboard and mouse events.

2. Convert them to environment actions.
3. Call `env.step()`.
4. Draw updated grid and HUD.
5. Refresh the screen.

Pygame requires this continuous loop to stay responsive. Without it, the window would freeze.

The GUI makes Shover-World much easier to debug, visually understandable, fun to interact with, and helpful for checking whether perfect squares, chain pushes, and special actions work.

Even though the GUI is not needed for reinforcement learning, it is extremely valuable for testing and showing the environment to others.

V. EXPERIMENTS AND OBSERVATIONS

After finishing both the environment and the GUI, I tested Shover-World in different ways to see whether everything worked correctly. I tried playing the game myself, and I also used some test units. These experiments helped me understand how the rules behave in real situations, which features are easy to use, and which ones need careful planning. Some common patterns I saw were:

- It wastes stamina by repeating invalid moves.
- It often tries to push boxes when the path is blocked.
- It rarely creates perfect squares by accident.
- It almost never reaches lava in a useful way.

Most random runs ended because the agent's stamina reached zero:

$$\text{stamina} \rightarrow 0.$$

Even though the agent played poorly, these experiments were still useful. They helped make sure the environment does not crash, even when actions happen quickly and randomly. Watching the random agent in the GUI also helped catch small mistakes. For example:

- Chain pushes worked correctly even if the chain was long.
- Stamina always changed by the right amount.
- Lava destruction refunded stamina as expected.

So even if the random agent “fails” the game, it was a good tool for stress-testing the environment.

The most helpful experiments came from playing the game manually in the GUI. As a human, I could purposely try different situations and see how the environment responded. Some of the things I noticed are listed below.

Chain pushing was easy to understand visually. When I pushed several boxes in a row, they moved smoothly together. If the push was illegal, the agent simply stayed in place. This made it easy to learn which pushes were allowed.

Perfect squares were fun to test manually. When I formed a clean 2×2 or 3×3 square, the environment detected it immediately. I could also see the age of the square increasing every step. Eventually, old squares dissolved:

$$\text{age} \geq \text{perf_sq_initial_age}.$$

VI. CONCLUSION AND FUTURE WORK

The Shover-World project helped me understand how a grid world can become complex even with simple rules. By combining movement, pushing boxes, stamina limits, perfect squares, and special actions, the environment became fun to test and interesting to play with. Building the system step by step also taught me how important it is to update the state correctly and make sure every rule works the same way every time.

Creating a GUI was very helpful because it allowed me to see what was happening in the environment. It made debugging easier and made the project feel more like a real game. The environment also follows the Gym API, which means it can be used later with reinforcement learning algorithms.